

성능 최적화

📅 날짜	@2025년 6월 20일
📌 선택	DL세션 과제
📌 주차	16주차
🌟 진행 상태	완료

8.1 성능 최적화

- 8.1.1 데이터를 사용한 성능 최적화
- 8.1.2 알고리즘을 이용한 성능 최적화
- 8.1.3 알고리즘 튜닝을 통한 성능 최적화
- 8.1.4 앙상블을 이용한 성능 최적화

8.2 하드웨어를 이용한 성능 최적화

- 8.2.1 CPU와 GPU 사용의 차이
- 8.2.2 GPU를 이용한 성능 최적화

8.3 하이퍼파라미터를 이용한 성능 최적화

- 8.3.1 배치 정규화를 이용한 성능 최적화
- 8.3.2 드롭아웃을 이용한 성능 최적화
- 실습
- 8.3.3 조기 종료를 이용한 성능 최적화



딥러닝 파이토치 교과서 8장

8.1 성능 최적화

8.1.1 데이터를 사용한 성능 최적화

: 많은 데이터를 수집하여 성능을 최적화함

- 최대한 많은 데이터 수집하기
- 데이터 수집이 어려운 경우, 임의로 데이터를 생성할 수 있음.
- 데이터 범위(scale) 조정하기
 - 활성화 함수로 sigmoid 사용 : 데이터셋 범위를 0~1

- 하이퍼볼릭 탄젠트 사용: -1~1 범위 데이터셋

8.1.2 알고리즘을 이용한 성능 최적화

유사한 용도의 알고리즘들을 선택하여 훈련시켜보고, 최적의 성능을 보이는 알고리즘을 선택

- 머신러닝의 데이터 분류: SVM, K-NN
- 시계열 데이터: RNN, LSTM, GRU 등

8.1.3 알고리즘 튜닝을 통한 성능 최적화

모델의 다양한 하이퍼파라미터를 변경하면서 훈련시키고, 최적의 성능을 도출

- **진단:** 성능 향상이 어느 순간 멈춘다면, 원인 분석이 필요 → 평과 결과를 바탕으로 과적합 인지 혹은 다른 원인으로 성능 향상에 문제가 있는지에 대한 insight를 얻음
- 훈련 성능 >>> 검증 성능 → 과적합 의심, 이를 해결하기 위해 규제화를 진행할 수 있음.
- 훈련 성능 & 검증 성능이 모두 좋지 않다면 과소적합을 의심할 수 있음 → 네트워크 구조 변경, 에포크 수 조정 등
- **가중치:** 가중치에 대한 초기값은 작은 난수 사용, 오토 인코더 같은 비지도 학습을 이용해 사전 훈련 후 지도 학습할 수 있음
- **학습률:** 모델의 네트워크 구성에 따라 다르므로 조금씩 변경해야 함.
 - 네트워크 계층이 많은 경우, 학습률을 낮아야
 - 네트워크 계층이 적은 경우, 학습률을 작게 설정
- **활성화 함수:** 활성화 함수를 변경할 때, 손실함수도 함께 변경해야 함
- **배치와 에포크:** 큰 에포크와 작은 배치를 사용하는 것이 최근 딥러닝 트렌드
 - 적절한 배치 크기를 위해 훈련 데이터셋의 크기를 동일하게 하거나, 하나의 배치로 훈련 시키는 등 다양한 테스트를 진행하는 것이 좋음.
- **옵티마이저 및 손실 함수:** 일반적으로 확률적 경사 하강법을 많이 사용
 - 아담(Adam), 알엠에스프롭(RMSProp) 등 좋은 성능을 보임.
 - 다양한 옵티마이저와 손실 함수 적용해보고 성능이 최고인 것을 선택해야 함.
- **네트워크 구성:** (= 네트워크 토폴로지)

- 최적의 네트워크는 쉽게 알기 어려워 네트워크 구성을 변경해가면서 성능을 테스트해야 함.
- 은닉층에 뉴런을 늘림(네트워크가 넓다)
- 네트워크 계층을 늘리되 뉴런 개수는 줄임(네트워크가 깊다)

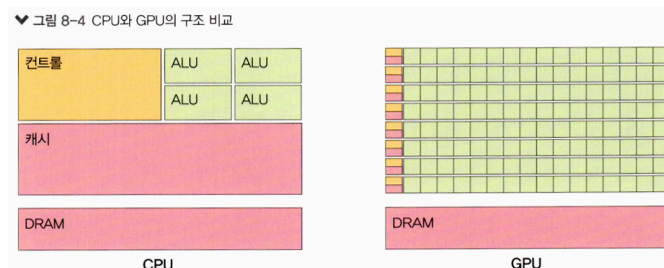
8.1.4 앙상블을 이용한 성능 최적화

모델을 두 개 이상 섞어서 사용하는 기법

- 알고리즘 튜닝을 위한 성능 최적화 방법은 하이퍼파라미터에 대한 경우의 수를 모두 고려해야 함 → 시행착오가 많이 필요

8.2 하드웨어를 이용한 성능 최적화

8.2.1 CPU와 GPU 사용의 차이



- CPU: 직렬 처리 방식
 - 연산을 담당하는 ALU
 - 명령어를 해석/실행하는 control
 - 데이터를 담아두는 cache
 - GPU: 병렬 처리를 위해 개발됨
 - ALU 개수가 많아짐
 - 캐시 메모리 비중은 낮음
- 연산을 수행하는 많은 ALU로 구성되어 있어 병렬 처리에 특화됨
- 코어에 ALU가 수백~수천 개 장착 → 3D 그래픽 작업 등을 빠르게 수행할 수 있음

개별적 코어 속도는 CPU >>> GPU

(ex1) 파이썬이나 매트랩처럼 행렬 연산을 많이 사용하는 재귀 연산이 대표적인 경우 '직렬' 연산 수행 → CPU가 적합

(ex2) 역전파처럼 복잡한 미분은 병렬 연산을 해야 속도가 빨라짐 → GPU가 적합

- 딥러닝은 방대한 데이터를 벡터로 변환한 후 연산을 수행함 ⇒ GPU 병렬 처리를 통해 모델 훈련 시간을 단축시킬 수 있음.

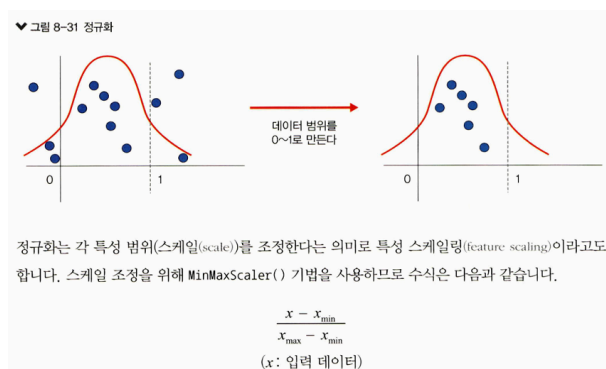
8.2.2 GPU를 이용한 성능 최적화

- 윈도우 환경에서 GPU용 파이토치를 설치하려면 CUDA와 cuDNN을 설치해야 함
- CUDA: NVIDIA에서 개발한 GPU 개발 툴로 많은 양의 연산을 동시에 처리할 수 있게 함
 - 딥러닝, 채굴과 같은 수학 연산에서 많이 사용

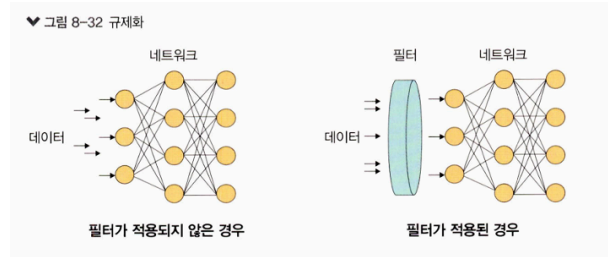
8.3 하이퍼파라미터를 이용한 성능 최적화

8.3.1 배치 정규화를 이용한 성능 최적화

- 정규화(regularization) = 특성 스케일링(feature scaling)
: 데이터를 사용자가 원하는 범위로 제한하는 것.



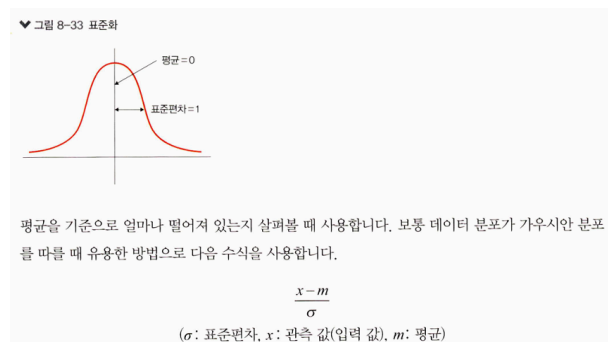
- 규제화(regularization)
: 모델 복잡도를 줄이기 위한 제약을 두는 방법
데이터가 네트워크에 들어가기전 필터를 적용하는 것과 유사



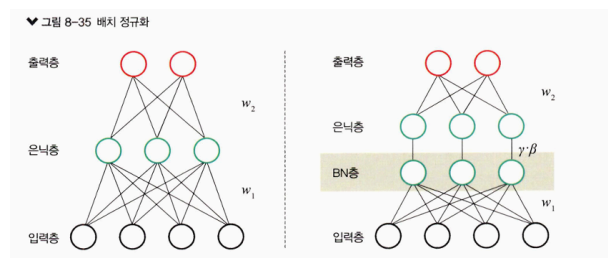
- 드롭아웃
- 조기 종료
- 표준화(standardization)

: 기존 데이터를 평균은 0, 표준편차는 1인 형태의 데이터로 만드는 방법

- 표준화 스칼라(standard scaler) 혹은 z-스코어 정규화(z-score normalization)



- 배치 정규화(batch normalization)
- : 기울기 소멸/기울기 폭발과 같은문제를 해결하기 위함.
- ReLU 사용, 초깃값 튜닝, 학습률 조정(lr)



공변량 변화(internal covariance shift) 때문에 기울기 소멸/폭발이 발생

→ 네트워크의 각 층마다 활성화 함수가 적용되면서 입력 값들의 분포가 계속 바뀌는 현상

- 분산된 분포를 정규 분포로 만들기 위해 표준화와 유사한 방식을 미니 배치에 적용
→ 평균은 0으로, 표준편차는 1로 유지하도록 함.

$$\begin{aligned}\mu\beta &\leftarrow \frac{1}{m} \sum_{i=1}^m x_i & \cdots \text{①} \\ \sigma^2\beta &\leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 & \cdots \text{②} \\ \hat{x}_i &\leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} & \cdots \text{③} \\ y_i &\leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) & \cdots \text{④}\end{aligned}$$

$\left(\begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)$

① 미니 배치 평균을 구합니다.
② 미니 배치의 분산과 표준편차를 구합니다.
③ 정규화를 수행합니다.
④ 스케일(scale)을 조정(데이터 분포 조정)합니다.

매 단계마다 활성화 함수를 거치면서 데이터셋 분포가 일정해지기 때문에 속도를 향상시킬 수 있음

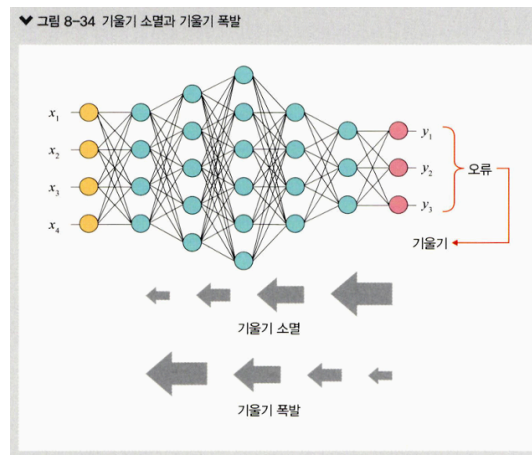
<단점>

1. 배치 크기가 작을 때는 정규화 값이 기존 값과 다른 방향으로 훈련될 수 있음
 - a. 분산이 0이면 정규화 자체가 안 되는 경우가 생길 수 있음
2. RNN은 네트워크 계층별로 미니 정규화를 적용해야 함. → 모델이 더 복잡해지면서 비효율적일 수 있음.
 - 배치 정규화 적용 시 성능 > 적용 X 성능 → 많이 사용



기울기 소멸& 기울기 폭발

- 기울기 소멸: 오차 정보를 역전파 시키는 과정에서 기울기가 급격히 0에 가까워져 학습이 되지 않는 현상
- 학습 과정에서 기울기가 급격히 커지는 현상



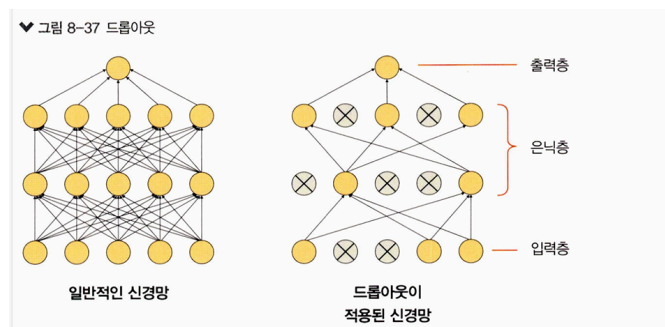
8.3.2 드롭아웃을 이용한 성능 최적화

- 과적합의 문제: 훈련 데이터셋을 과하게 학습하여 테스트 데이터셋에 대한 오류가 증가하는 것



dropout

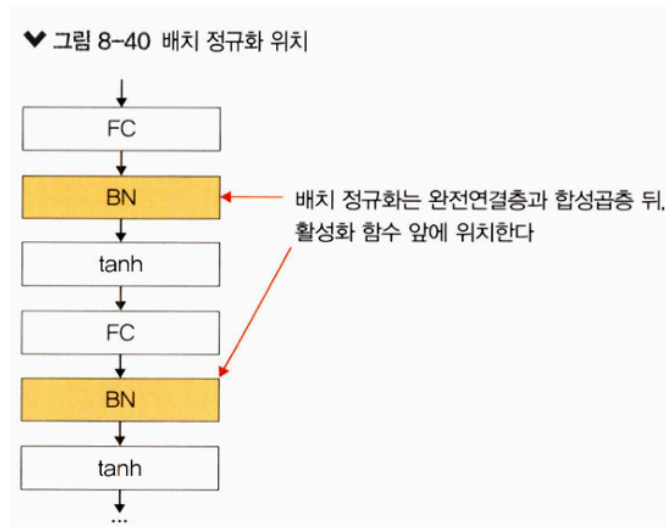
훈련할 때 일정 비율의 뉴런만 사용하고, 나머지 뉴런에 대한 가중치를 업데이트 하지 않음



- 학습시마다 무작위로 어떤 노드들을 비활성화
- 테스트 데이터로 평가할 때는 노드들을 모두 사용하여 출력하되, 노드 삭제 비율(드롭아웃 비율)을 곱해서 성능을 평가

실습

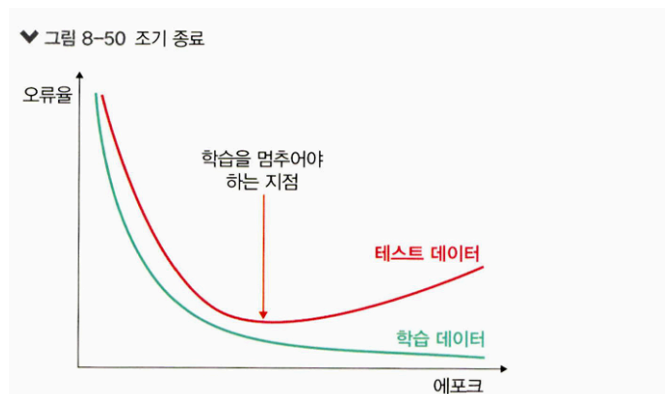
배치 정규화: 은닉층에서 학습이 진행되 르때마다 입력 분포가 변하면서 가중치가 엉뚱한 방향으로 갱신되는 문제 발생 → 고르게 분포를 보정



8.3.3 조기 종료를 이용한 성능 최적화

조기종료(early stopping): 뉴럴 네트워크가 과적합을 피하는 규제 기법

- 훈련 데이터와 별도로 검증 데이터 준비
- 매 에포크마다 검증 데이터에 대한 오차를 측정하여 모델 종료 시점을 제어 → 과적합이 발생하기 전까지 학습



- 조기 종료는 학습을 언제 종료시킬지 결정할 뿐이지, 최고의 성능을 갖는 모델을 보장하지 않음.